Smart Notes App

Proyecto IHC

Aaron Rodrigo Ramos Reyes
Universidad Autónoma Metropolitana - Unidad Cuajimalpa

13 de febrero de 2026

Índice

1. Introducción **3**

2. Tecnologías utilizadas **3**

3. Arquitectura del proyecto **3**
3.1. Diagrama de clases 4
3.2. Diagrama de flujo (agregar nota) 5
3.3. Código PlantUML 5

4. Flujo de uso **7**
4.1. Captura: Pantalla principal 7
4.2. Captura: Crear/Editar nota 8
4.3. Captura: Detalle de grupo 9
4.4. Captura: Área de borrado por arrastre 10
4.5. Captura: Menu nota/carpeta 11
4.6. Captura: Menu nota 12
4.7. Captura: Menu carpeta 13

5. Funciones principales **13**

6. Áreas a mejorar para la próxima entrega **14**
6.1. Integración de Inteligencia Artificial 14
6.2. Comodities y experiencia de usuario 14

7. Conclusiones **15**

1. Introducción

Smart Notes es una aplicación móvil desarrollada en Flutter que permite gestionar notas personales organizadas en grupos (carpetas). El proyecto implementa funcionalidades de arrastrar y soltar, personalización de colores, historial de edición (Undo/Redo) y persistencia local mediante Hive. Este documento describe la arquitectura, tecnologías empleadas, flujo de uso, funciones implementadas y las áreas de mejora previstas para la próxima entrega.

2. Tecnologías utilizadas

- **Lenguaje:** Dart 3.x
- **Framework:** Flutter 3.x
- **Persistencia local:** Hive + Hive Flutter (NoSQL, rápido y nativo)
- **Generación de código:** build_runner, hive_generator
- **IDE:** Visual Studio Code
- **Control de versiones:** Git

3. Arquitectura del proyecto

La aplicación sigue una arquitectura basada en capas simplificada, típica de Flutter:

- **Capa de presentación:** Widgets (screens, diálogos, widgets reutilizables).
- **Capa de lógica de negocio:** Servicios (NoteService, GroupService) y manejo de estado con ValueNotifier + ValueListenableBuilder.
- **Capa de datos:** Modelos (Note, NoteGroup) y adaptadores Hive.

3.1. Diagrama de clases

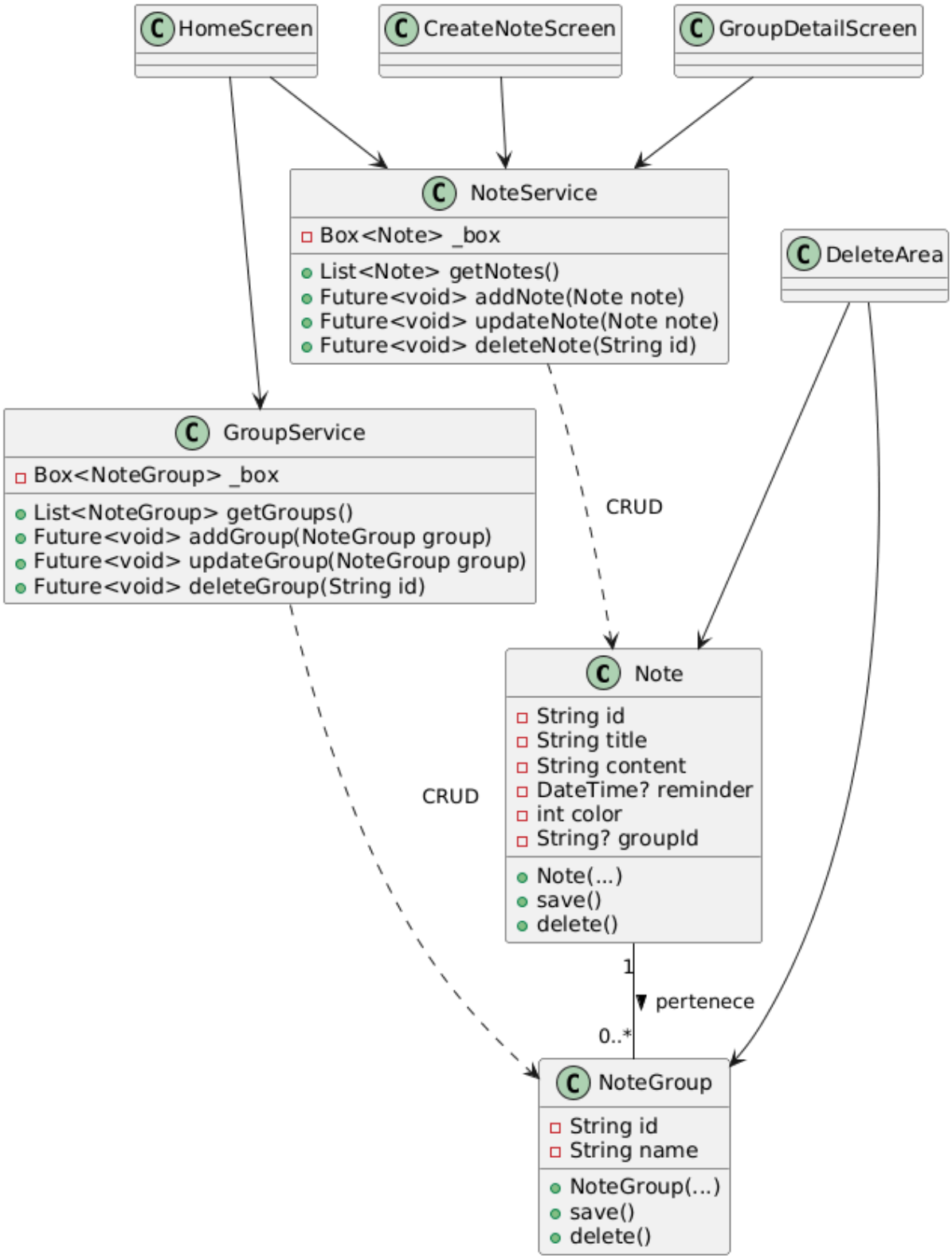

Figura 1: Diagrama de clases UML (generado con PlantUML). Muestra las relaciones entre modelos, servicios y widgets principales.

3.2. Diagrama de flujo (agregar nota)

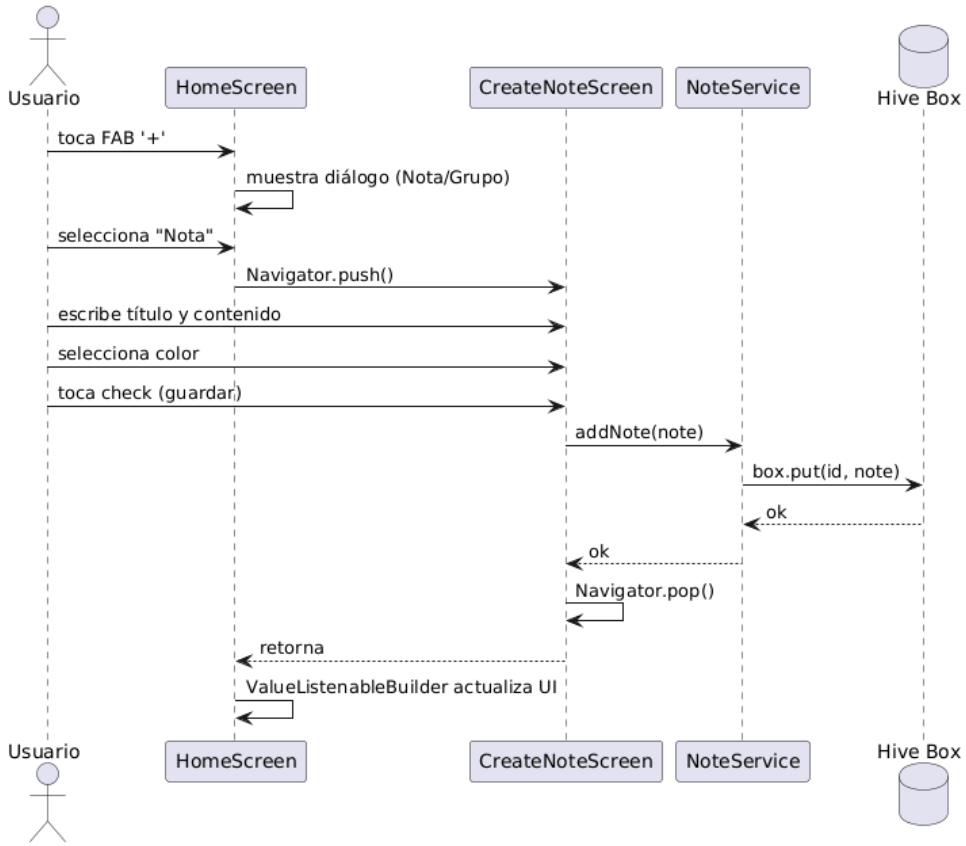

Figura 2: Flujo de creación de una nota desde el botón flotante hasta la persistencia en Hive.

3.3. Código PlantUML

Para generar los diagramas anteriores, utiliza el siguiente código en PlantUML:

Diagrama de clases (clases.puml)

```
@startuml
class Note {
  - String id
  - String title
  - String content
  - DateTime? reminder
  - int color
  - String? groupId
  + Note(...)
  + save()
  + delete()
}
class NoteGroup {
  - String id
  - String name
  + NoteGroup(...)
  + save()
```

```
  + delete()
}
class NoteService {
  - Box<Note> _box
  + List<Note> getNotes()
  + Future<void> addNote(Note note)
  + Future<void> updateNote(Note note)
  + Future<void> deleteNote(String id)
}
class GroupService {
  - Box<NoteGroup> _box
  + List<NoteGroup> getGroups()
  + Future<void> addGroup(NoteGroup group)
  + Future<void> updateGroup(NoteGroup group)
  + Future<void> deleteGroup(String id)
}
class HomeScreen
class CreateNoteScreen
class GroupDetailScreen
class DeleteArea

Note "1" -- "0..*" NoteGroup : pertenece >
NoteService ..> Note : CRUD
GroupService ..> NoteGroup : CRUD
HomeScreen --> NoteService
HomeScreen --> GroupService
GroupDetailScreen --> NoteService
CreateNoteScreen --> NoteService
DeleteArea --> Note
DeleteArea --> NoteGroup
@enduml
```

Diagrama de flujo (flujo_agregar_nota.puml)

```
@startuml
actor Usuario
participant "HomeScreen" as H
participant "CreateNoteScreen" as C
participant "NoteService" as S
database "Hive Box" as DB

Usuario -> H: toca FAB '+'
H -> H: muestra dialogo (Nota/Grupo)
Usuario -> H: selecciona "Nota"
H -> C: Navigator.push()
Usuario -> C: escribe titulo y contenido
Usuario -> C: selecciona color
Usuario -> C: toca check (guardar)
C -> S: addNote(note)
S -> DB: box.put(id, note)
DB --> S: ok
S --> C: ok
C -> C: Navigator.pop()
C --> H: retorna
H -> H: ValueListenableBuilder actualiza UI
@enduml
```

4. Flujo de uso

A continuación se describen las interacciones principales del usuario con la aplicación.

4.1. Captura: Pantalla principal

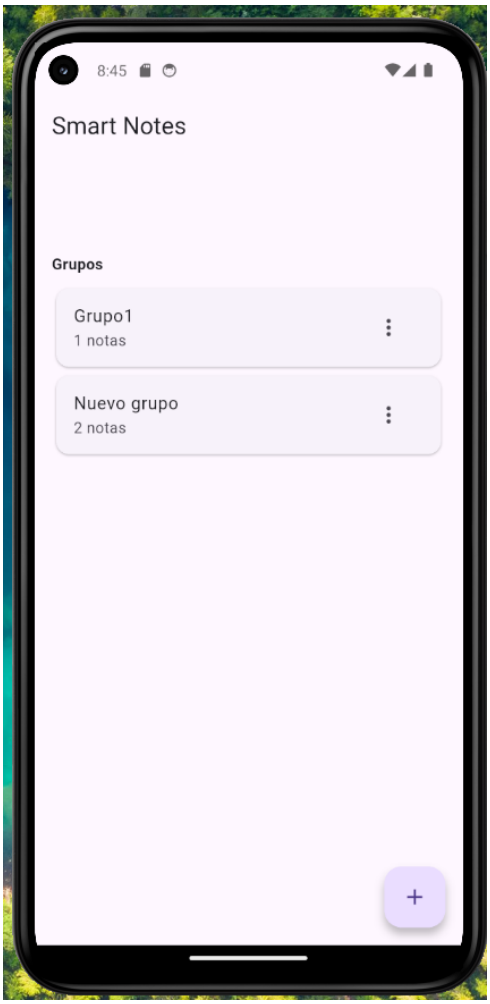

Figura 3: Pantalla principal (HomeScreen). Muestra listado de notas sin grupo y grupos.

4.2. Captura: Crear/Editar nota

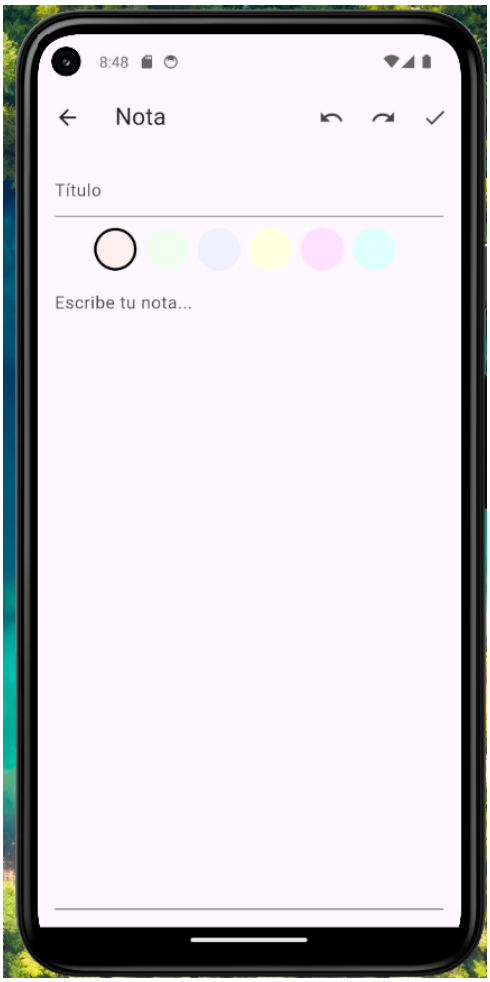

Figura 4: Pantalla de creación/edición de nota. Incluye selector de colores pastel y botones Undo/Redo.

4.3. Captura: Detalle de grupo

Figura 5: Pantalla de detalle de grupo. Muestra las notas pertenecientes y menú para editar/eliminar el grupo.

4.4. Captura: Área de borrado por arrastre

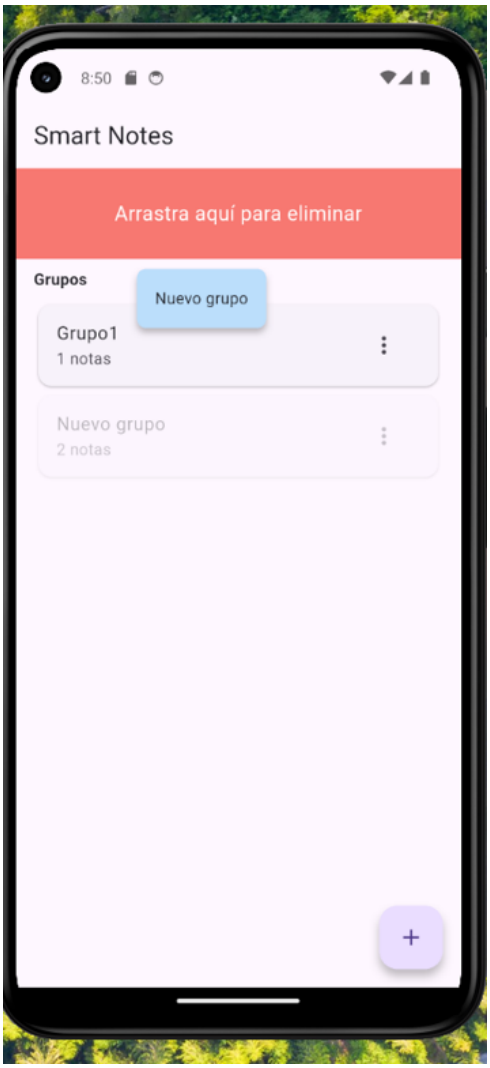

Figura 6: Área superior que aparece al arrastrar una nota o grupo; soltar allí elimina el elemento previa confirmación.

4.5. Captura: Menu nota/carpeta

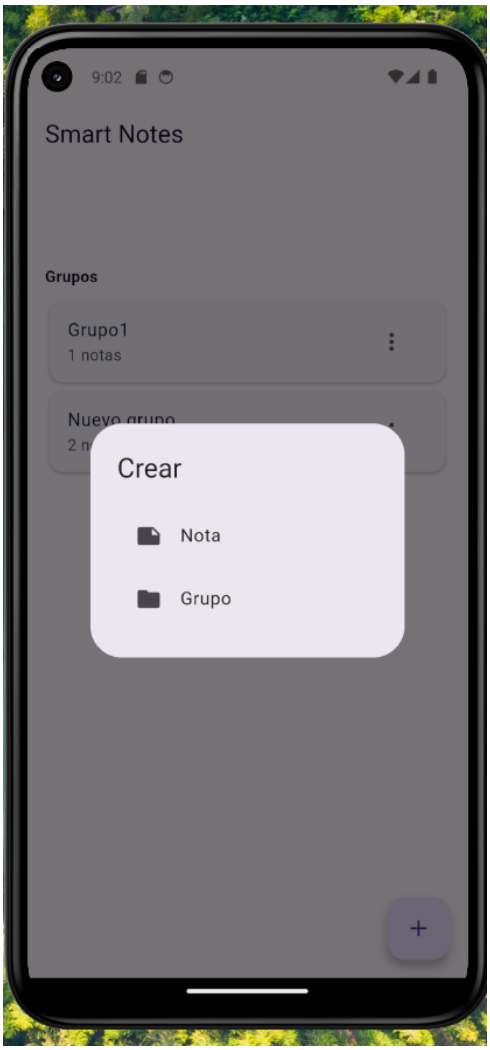

Figura 7: Menú contextual que permite la creacion de notas o grupos.

4.6. Captura: Menu nota

Figura 8: Menú contextual que permite eliminar, editar o agrupar notas.

4.7. Captura: Menu carpeta

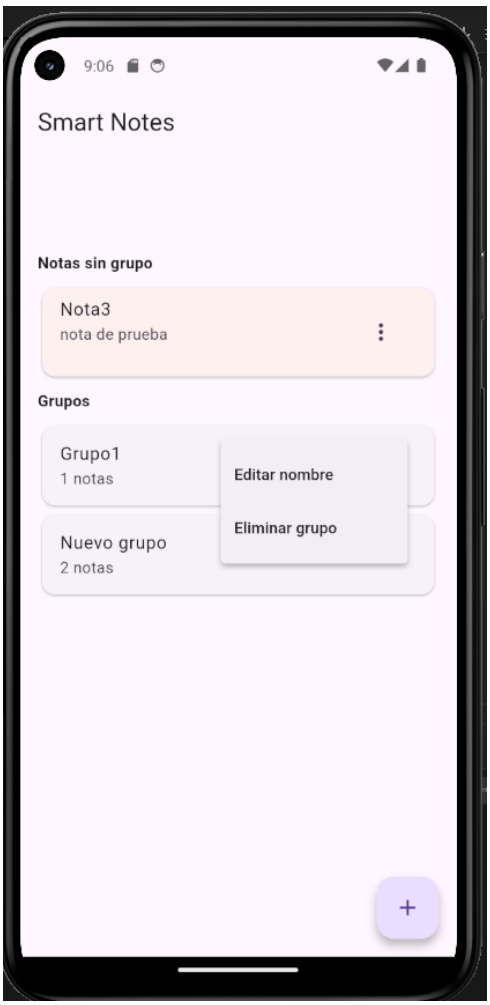

Figura 9: Menú contextual de una carpeta. Muestra opciones de edición, eliminación

5. Funciones principales

- **Creación de notas:** título, contenido, color pastel, recordatorio (opcional).
- **Edición de notas:** modificar cualquier campo, con historial de contenido (Undo/Redo).
- **Eliminación:** mediante botón en menú contextual o arrastrando al área de borrado (siempre con confirmación).
- **Grupos (carpetas):** crear, editar nombre, eliminar (las notas se desasignan automáticamente).
- **Asignación de notas a grupos:** desde el menú de tres puntos en cada nota o arrastrando una nota sobre un grupo.
- **Arrastrar y soltar:**
  - Nota sobre otra nota → crea un grupo automático con ambas.

    - Nota sobre un grupo → asigna la nota a ese grupo.
    - Nota/grupo al área superior → elimina (confirmación).

- **Personalización:** 6 colores pastel para el fondo de las notas.
- **Persistencia:** todos los datos se guardan localmente con Hive.

6. Áreas a mejorar para la próxima entrega

Tras una primera ronda de levantamiento de requerimientos, se han identificado las siguientes mejoras y nuevas funcionalidades:

6.1. Integración de Inteligencia Artificial

- **Reconocimiento de texto (OCR):** el usuario podrá tomar una foto o seleccionar una imagen de la galería y extraer el texto para convertirlo automáticamente en una nota.
- **Reconocimiento de voz:** dictado de notas mediante micrófono, transcribiendo el audio a texto en tiempo real.
- **Sugerencias inteligentes:** basado en el contenido de la nota, la IA propondrá:
    - Creación de una carpeta temática (ej. si la nota habla de "recetas", sugerirá crear o añadir a una carpeta "Cocina").
    - Establecer recordatorios automáticos detectando fechas, horas o frases como "mañana", "próximo lunes".

6.2. Comodities y experiencia de usuario

- **Arrastrar nota a carpeta:** actualmente es posible arrastrar una nota sobre otra y formar un grupo automático sin embargo se busca que al arrastar una nota a un grupo ya establecido se agrege de manera automática.
- **Selector de color global:** al elegir un color para una nota, la pantalla de edición adoptará temporalmente (mientras se encuentre dento de la nota) ese color de fondo para ofrecer una vista previa inmersiva.
- **Selección múltiple:** manteniendo presionada una nota (sin arrastrar) se activará un modo de selección que permita marcar varias notas. Las acciones disponibles serán:
    - Eliminar todas las seleccionadas.
    - Asignar todas a un mismo grupo.
    - Crear un nuevo grupo con todas las seleccionadas.
- **Otros comodities:** búsqueda global de notas.

7. Conclusiones

Smart Notes ha logrado implementar un sistema funcional de notas con organización por grupos, interacciones táctiles avanzadas (drag & drop, menús contextuales) y una interfaz limpia y personalizable. La arquitectura basada en Hive garantiza rapidez y facilidad de mantenimiento. Las mejoras propuestas elevarán la aplicación a un nivel superior, haciéndola más inteligente y cómoda para el usuario final.